

UNIVERSIDAD AUTÓNOMA DE BUCARAMANGA

Facultad de Ingeniería

Ingeniería de Sistemas

INFORME DE LABORATORIO 2

Flota IoT Central con generación de datos Python en máquina virtual y ESP32 Arduino en Wokwi

Presentado por:

Juan Sebastian Lizcano Jaimes

Angie Katherine Suarez Ortiz

IoT + Cloud + Sistemas Distribuidos

Bucaramanga, septiembre de 2026

Resumen ejecutivo

Se reutilizó la aplicación y la plantilla del Laboratorio 1 para construir una flota de ocho dispositivos en Azure IoT Central: seis simulados, un cliente Python ejecutado en una máquina virtual Debian 12 y un ESP32 virtual en Wokwi. Los nodos de código publicaron *PotenciaGenerada*, *TemperaturaDelPanel*, *VoltajeDeSalida* y *CorrienteDeSalida*, y procesaron comandos remotos.

Objetivos

- Reutilizar una sola plantilla para ocho dispositivos.
- Generar telemetría desde Python dentro de una VM.
- Implementar un ESP32 virtual con DHT22, potenciómetro y LED.
- Validar DPS, MQTT/TLS, telemetría, comandos y una regla por umbral.
- Comparar los tres mecanismos de generación de datos.

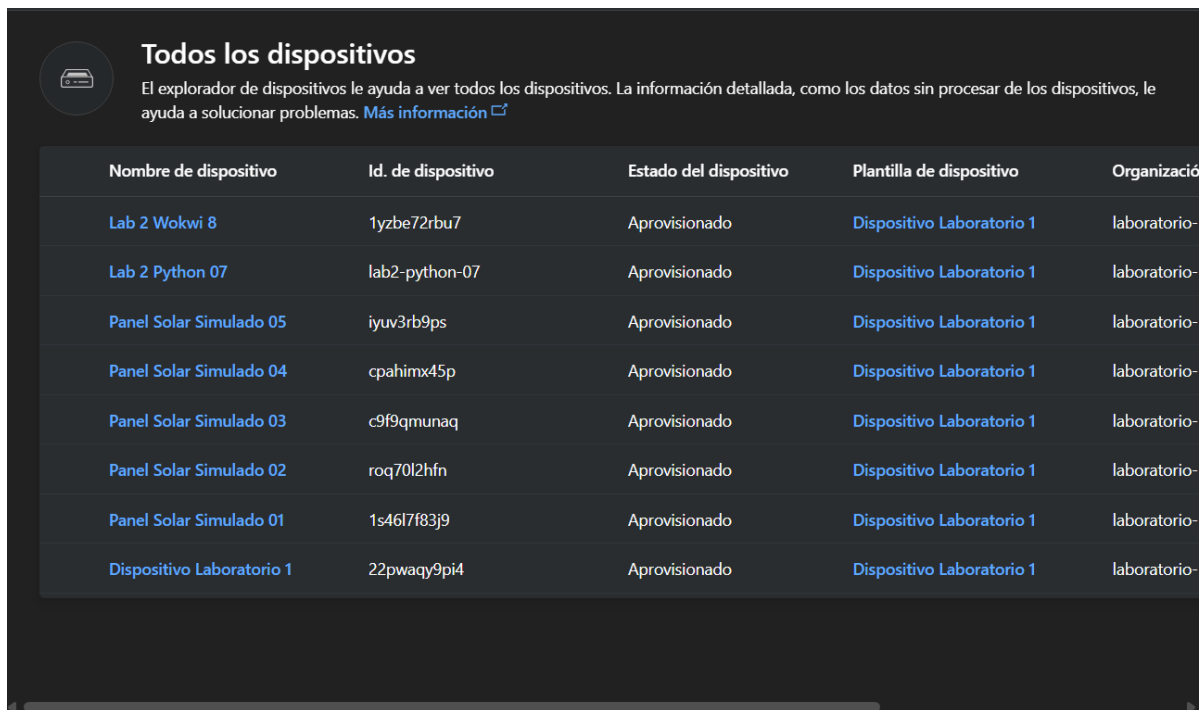
1. Plantilla e inventario de la flota

La plantilla *Dispositivo Laboratorio 1* modeló un panel solar con cuatro telemetrías, propiedades descriptivas, *EstadoDeAlerta* y los comandos *ReiniciarControlador* y *ControlarLEDDeAlerta*. Los nombres internos se mantuvieron idénticos en Python y en el firmware del ESP32.

The image shows a screenshot of the Azure IoT Central template configuration interface. It displays two sections: 'Estado de alerta' and 'Controlar LED de alerta'. The 'Estado de alerta' section includes fields for 'Esquema' (Boolean), 'Grabable' (Activado), 'Valor inicial' (Falso), 'Color', 'Nombre true', 'Nombre false', 'Unidad' (Ninguno), 'Unidad de visualización', 'Comentario', and 'Descripción' (indica si el panel solar esta en estado de alerta). The 'Controlar LED de alerta' section includes fields for 'Agregar a la cola si está sin conexión' (Desactivar), 'Solicitud' (Activado), 'Nombre para mostrar' (Estado del LED), 'Nombre' (EstadoDelLED), 'Esquema' (Boolean), 'Valor inicial' (Falso), 'Comentario', 'Descripción', 'Respuesta' (Activado), 'Nombre para mostrar' (Resultado), 'Nombre' (Resultado), 'Esquema' (String), and 'Comentario'.

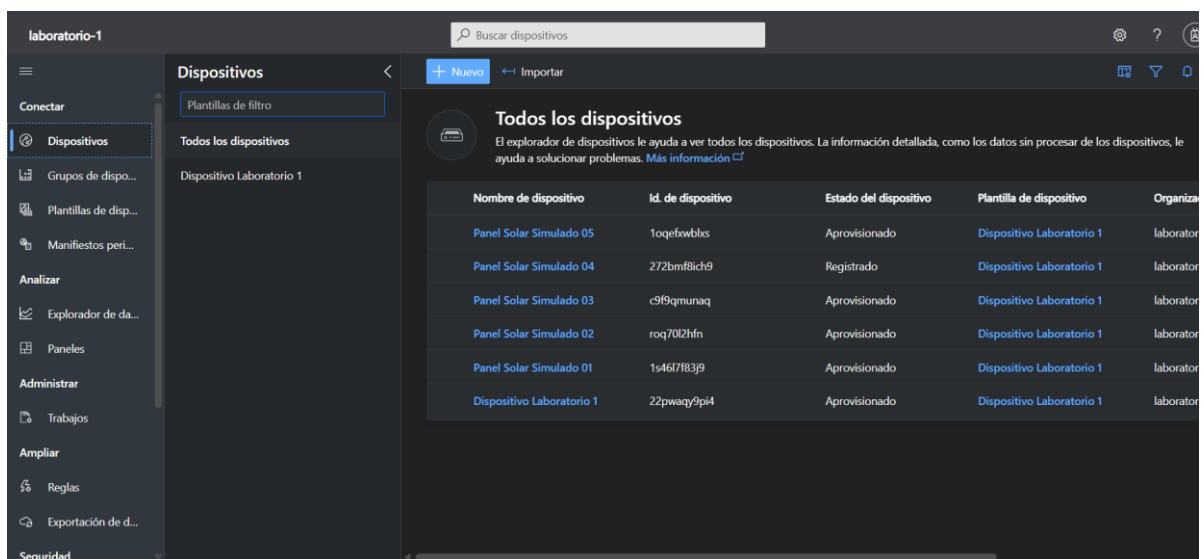
Figura 1. Propiedad *EstadoDeAlerta* y comando *ControlarLEDDeAlerta* definidos en la plantilla.

La flota final quedó conformada por seis paneles simulados, Lab 2 Python 07 y Lab 2 Wokwi 8.



Nombre de dispositivo	Id. de dispositivo	Estado del dispositivo	Plantilla de dispositivo	Organización
Lab 2 Wokwi 8	1yzbe72rbu7	Aprovisionado	Dispositivo Laboratorio 1	laboratorio-
Lab 2 Python 07	lab2-python-07	Aprovisionado	Dispositivo Laboratorio 1	laboratorio-
Panel Solar Simulado 05	iyuv3rb9ps	Aprovisionado	Dispositivo Laboratorio 1	laboratorio-
Panel Solar Simulado 04	cpahimx45p	Aprovisionado	Dispositivo Laboratorio 1	laboratorio-
Panel Solar Simulado 03	c9f9qmunaq	Aprovisionado	Dispositivo Laboratorio 1	laboratorio-
Panel Solar Simulado 02	roq70l2hfn	Aprovisionado	Dispositivo Laboratorio 1	laboratorio-
Panel Solar Simulado 01	1s46l7f83j9	Aprovisionado	Dispositivo Laboratorio 1	laboratorio-
Dispositivo Laboratorio 1	22pwaqy9pi4	Aprovisionado	Dispositivo Laboratorio 1	laboratorio-

Figura 2. Inventario completo de ocho dispositivos.



Nombre de dispositivo	Id. de dispositivo	Estado del dispositivo	Plantilla de dispositivo	Organización
Panel Solar Simulado 05	1oqefwblbs	Aprovisionado	Dispositivo Laboratorio 1	laboratorio-
Panel Solar Simulado 04	272bmf8ich9	Registrado	Dispositivo Laboratorio 1	laboratorio-
Panel Solar Simulado 03	c9f9qmunaq	Aprovisionado	Dispositivo Laboratorio 1	laboratorio-
Panel Solar Simulado 02	roq70l2hfn	Aprovisionado	Dispositivo Laboratorio 1	laboratorio-
Panel Solar Simulado 01	1s46l7f83j9	Aprovisionado	Dispositivo Laboratorio 1	laboratorio-
Dispositivo Laboratorio 1	22pwaqy9pi4	Aprovisionado	Dispositivo Laboratorio 1	laboratorio-

Figura 3. Vista general de dispositivos y estados de aprovisionamiento.

2. Dispositivos simulados y panel general

Los seis dispositivos simulados utilizaron el generador nativo de IoT Central. Las evidencias de esta sección corresponden únicamente a simuladores y al panel general.

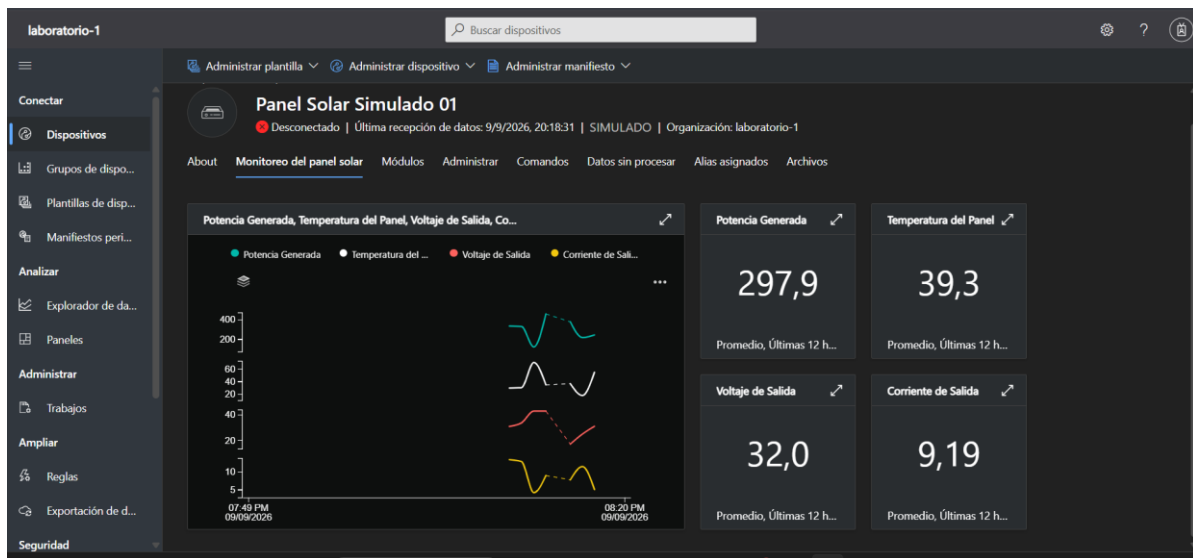


Figura 4. Panel Solar Simulado 01 con telemetría activa.

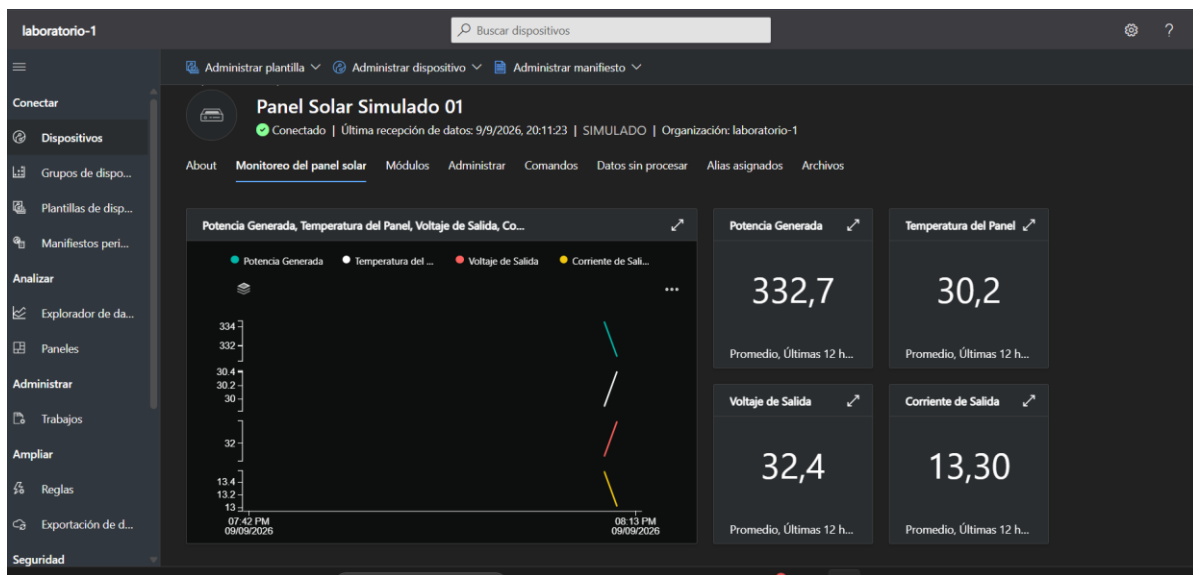


Figura 5. Panel Solar Simulado 6 con telemetría activa.

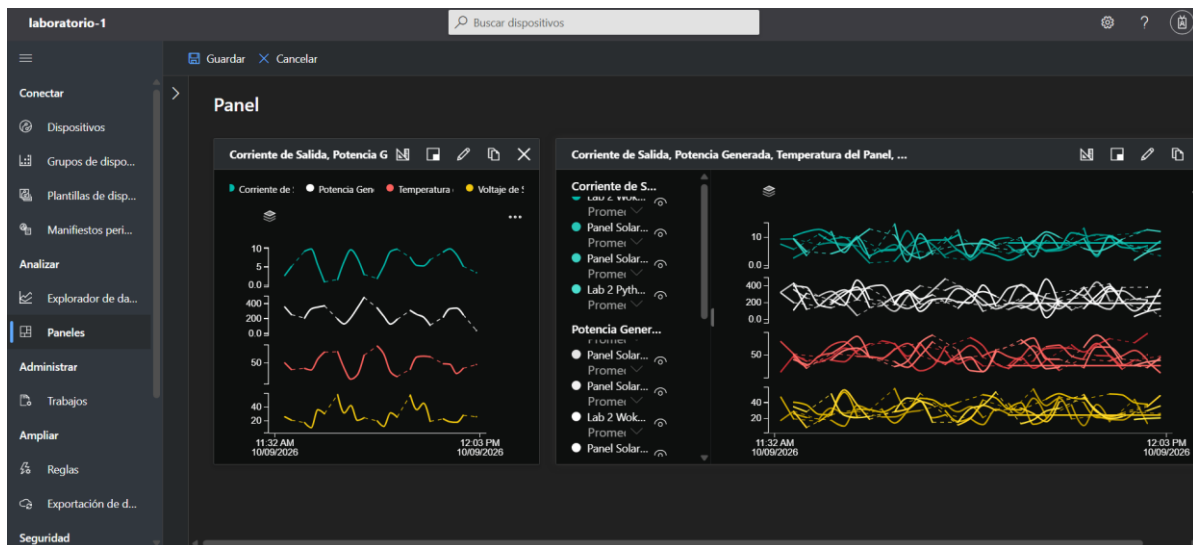


Figura 6. Panel general de supervisión de la aplicación.

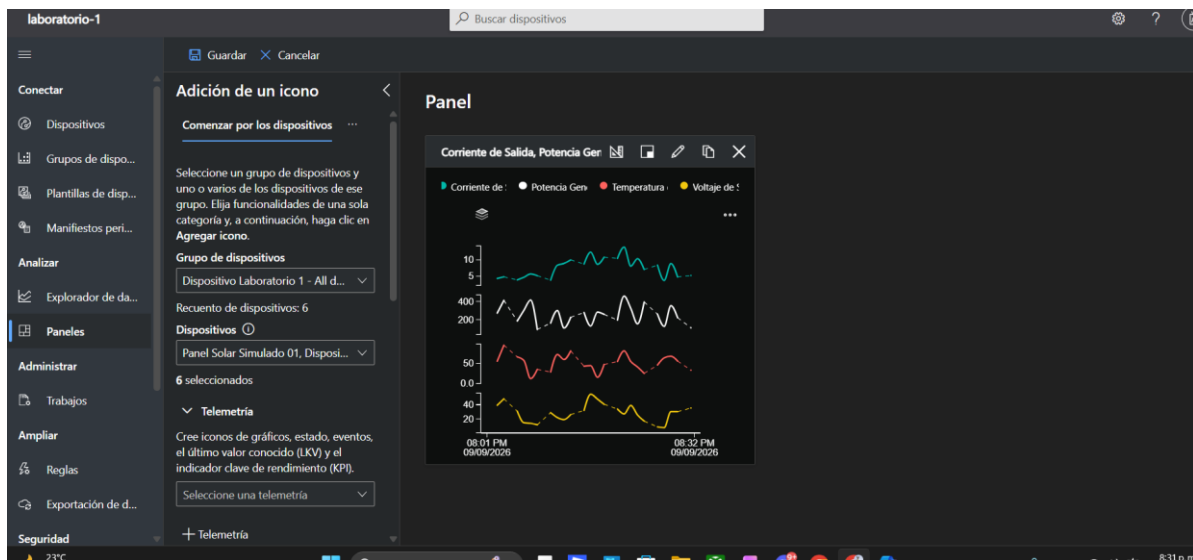


Figura 7. Edición del panel general y selección de capacidades.

3. Preparación de la máquina virtual

El nodo Python se ejecutó en una VM de Azure con Debian GNU/Linux 12. El acceso se realizó mediante SSH desde PowerShell. Se instalaron Python, pip, venv, Git, curl, certificados y herramientas de compilación.



Figura 8. Recurso de máquina virtual utilizado para el laboratorio.

```
Debian GNU/Linux comes with ABSOLUTELY NO WARRANTY, to the extent
permitted by applicable law.
lab2@Laboratorio-2:~$ whoami
lab2
lab2@Laboratorio-2:~$ hostname
Laboratorio-2
lab2@Laboratorio-2:~$
```

Figura 9. Sesión SSH en Debian 12 y comprobación del host.

```
lab2@Laboratorio-2:~$ sudo apt install -y python3 python3-pip python3-venv python3-dev git curl ca-certificates build-essential nano
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
python3 is already the newest version (3.11.2-1+b1).
python3 set to manually installed.
curl is already the newest version (7.88.1-10+deb12u15).
ca-certificates is already the newest version (20250419~deb12u1).
nano is already the newest version (7.2-1+deb12u1).
```

Figura 10. Instalación de herramientas y dependencias en Debian.

```
(.env) lab2@Laboratorio-2:~/laboratorio2/python-vm$ python -m pip show azure-iot-device
Name: azure-iot-device
Version: 2.14.0
Summary: Microsoft Azure IoT Device Library
Home-page: https://github.com/Azure/azure-iot-sdk-python/
Author: Microsoft Corporation
Author-email: opensource@microsoft.com
License: MIT License
Location: /home/lab2/laboratorio2/python-vm/.venv/lib/python3.11/site-packages
Requires: deprecation, janus, paho-mqtt, PySocks, requests, requests-unixsocket2, typing-extensions, urllib3
Required-by:
(.env) lab2@Laboratorio-2:~/laboratorio2/python-vm$
```

Figura 11. Verificación de azure-iot-device en el entorno virtual.

4. Nodo Python en la VM

El dispositivo Lab 2 Python 07 se creó como dispositivo no simulado dentro de la plantilla. Las credenciales se almacenaron en .env, excluido del repositorio, y se comprobó su carga sin revelar la clave.

Crear un nuevo dispositivo

Para crear un dispositivo, seleccione una plantilla de dispositivo, un nombre y un id. único. [Más información](#)

Nombre de dispositivo *

Lab 2 Python 07

Id. de dispositivo *

222fzmnzq2w

Organización *

laboratorio-1

Plantilla de dispositivo *

Dispositivo Laboratorio 1

¿Quiere simular este dispositivo?
Un dispositivo simulado genera una telemetría que le permite probar el comportamiento de la aplicación antes de conectar un dispositivo real.

☐ No

¿Es un dispositivo de Azure IoT Edge?
Azure IoT Edge mueve el análisis en la nube y la lógica de negocios personalizada de la nube a sus dispositivos.

☐ Sí

Manifiesto perimetral

Seleccionar un manifiesto perimetral

Crear Cancelar

Figura 12. Creación de Lab 2 Python 07 en IoT Central.

```
(.venv) lab2@Laboratorio-2:~/laboratorio2/python-vm$ python - <<'PY'
import os
from dotenv import load_dotenv

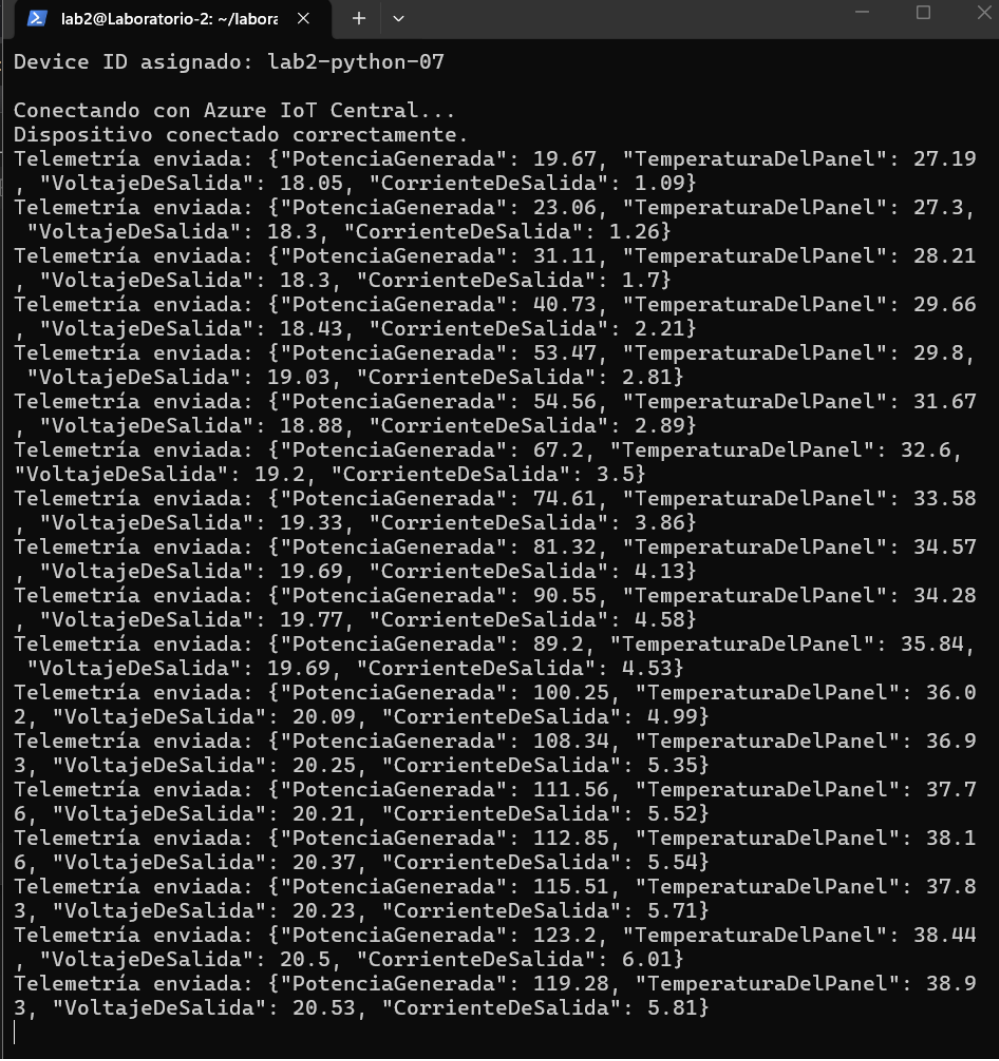
loaded = load_dotenv(dotenv_path=".env")

key = os.getenv("IOTC_DEVICE_KEY")

print("Archivo .env cargado:", loaded)
print("ID Scope:", os.getenv("IOTC_ID_SCOPE"))
print("Device ID:", os.getenv("IOTC_DEVICE_ID"))
print("Clave cargada:", bool(key))
print("Longitud de la clave:", len(key or ""))
PY
Archivo .env cargado: True
ID Scope: 0ne00FE096B
Device ID: 222fzmnzq2w
Clave cargada: True
Longitud de la clave: 44
(.venv) lab2@Laboratorio-2:~/laboratorio2/python-vm$
```

Figura 13. Validación segura de las variables del archivo .env.

El script realizó el aprovisionamiento mediante DPS y envió un JSON cada cinco segundos. La potencia se calculó a partir del voltaje y la corriente, mientras que la temperatura siguió una variación sintética coherente.



```
lab2@Laboratorio-2: ~/labori x + v
Device ID asignado: lab2-python-07

Conectando con Azure IoT Central...
Dispositivo conectado correctamente.
Telemetría enviada: {"PotenciaGenerada": 19.67, "TemperaturaDelPanel": 27.19, "VoltajeDeSalida": 18.05, "CorrienteDeSalida": 1.09}
Telemetría enviada: {"PotenciaGenerada": 23.06, "TemperaturaDelPanel": 27.3, "VoltajeDeSalida": 18.3, "CorrienteDeSalida": 1.26}
Telemetría enviada: {"PotenciaGenerada": 31.11, "TemperaturaDelPanel": 28.21, "VoltajeDeSalida": 18.3, "CorrienteDeSalida": 1.7}
Telemetría enviada: {"PotenciaGenerada": 40.73, "TemperaturaDelPanel": 29.66, "VoltajeDeSalida": 18.43, "CorrienteDeSalida": 2.21}
Telemetría enviada: {"PotenciaGenerada": 53.47, "TemperaturaDelPanel": 29.8, "VoltajeDeSalida": 19.03, "CorrienteDeSalida": 2.81}
Telemetría enviada: {"PotenciaGenerada": 54.56, "TemperaturaDelPanel": 31.67, "VoltajeDeSalida": 18.88, "CorrienteDeSalida": 2.89}
Telemetría enviada: {"PotenciaGenerada": 67.2, "TemperaturaDelPanel": 32.6, "VoltajeDeSalida": 19.2, "CorrienteDeSalida": 3.5}
Telemetría enviada: {"PotenciaGenerada": 74.61, "TemperaturaDelPanel": 33.58, "VoltajeDeSalida": 19.33, "CorrienteDeSalida": 3.86}
Telemetría enviada: {"PotenciaGenerada": 81.32, "TemperaturaDelPanel": 34.57, "VoltajeDeSalida": 19.69, "CorrienteDeSalida": 4.13}
Telemetría enviada: {"PotenciaGenerada": 90.55, "TemperaturaDelPanel": 34.28, "VoltajeDeSalida": 19.77, "CorrienteDeSalida": 4.58}
Telemetría enviada: {"PotenciaGenerada": 89.2, "TemperaturaDelPanel": 35.84, "VoltajeDeSalida": 19.69, "CorrienteDeSalida": 4.53}
Telemetría enviada: {"PotenciaGenerada": 100.25, "TemperaturaDelPanel": 36.02, "VoltajeDeSalida": 20.09, "CorrienteDeSalida": 4.99}
Telemetría enviada: {"PotenciaGenerada": 108.34, "TemperaturaDelPanel": 36.93, "VoltajeDeSalida": 20.25, "CorrienteDeSalida": 5.35}
Telemetría enviada: {"PotenciaGenerada": 111.56, "TemperaturaDelPanel": 37.76, "VoltajeDeSalida": 20.21, "CorrienteDeSalida": 5.52}
Telemetría enviada: {"PotenciaGenerada": 112.85, "TemperaturaDelPanel": 38.16, "VoltajeDeSalida": 20.37, "CorrienteDeSalida": 5.54}
Telemetría enviada: {"PotenciaGenerada": 115.51, "TemperaturaDelPanel": 37.83, "VoltajeDeSalida": 20.23, "CorrienteDeSalida": 5.71}
Telemetría enviada: {"PotenciaGenerada": 123.2, "TemperaturaDelPanel": 38.44, "VoltajeDeSalida": 20.5, "CorrienteDeSalida": 6.01}
Telemetría enviada: {"PotenciaGenerada": 119.28, "TemperaturaDelPanel": 38.93, "VoltajeDeSalida": 20.53, "CorrienteDeSalida": 5.81}
```

Figura 14. Ejecución del script Python y envío de las cuatro telemetrías.

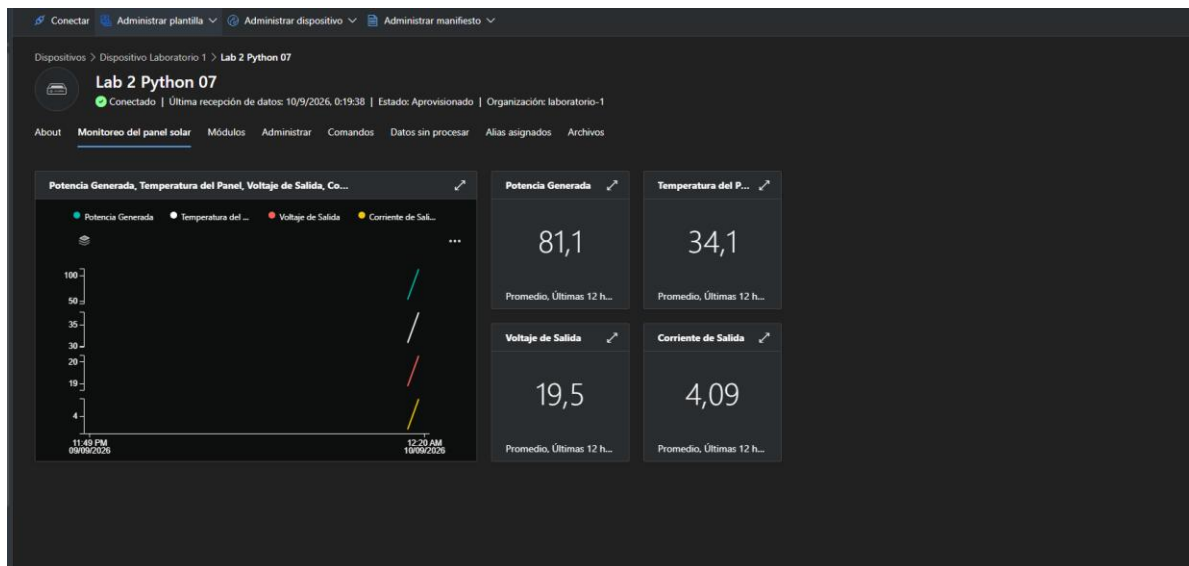


Figura 15. Panel correspondiente exclusivamente a Lab 2 Python 07.

4.1 Comando del nodo Python

ReiniciarControlador se ejecutó desde IoT Central. El script recibió el método directo, simuló el reinicio lógico y respondió con estado satisfactorio.

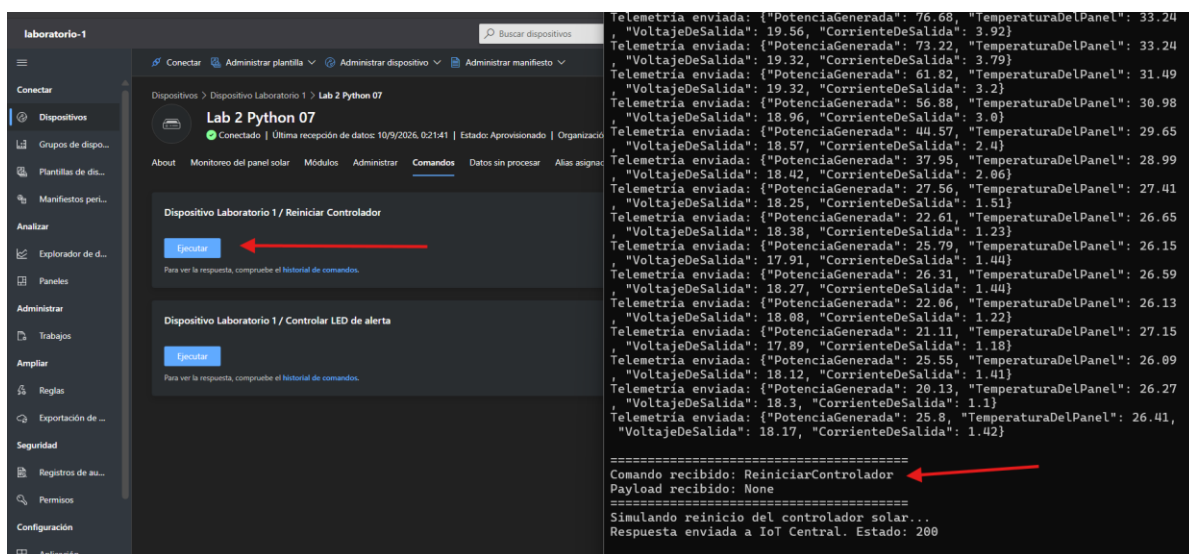


Figura 16. ReiniciarControlador sobre Lab 2 Python 07 y respuesta en la terminal.

5. Nodo ESP32 virtual en Wokwi

El nodo Wokwi se desarrolló por separado con PlatformIO. El circuito incorporó un ESP32, DHT22 en GPIO 15, potenciómetro en GPIO 34 y LED con resistencia en GPIO 2. El dispositivo utilizó Wokwi-GUEST, DPS y MQTT/TLS.

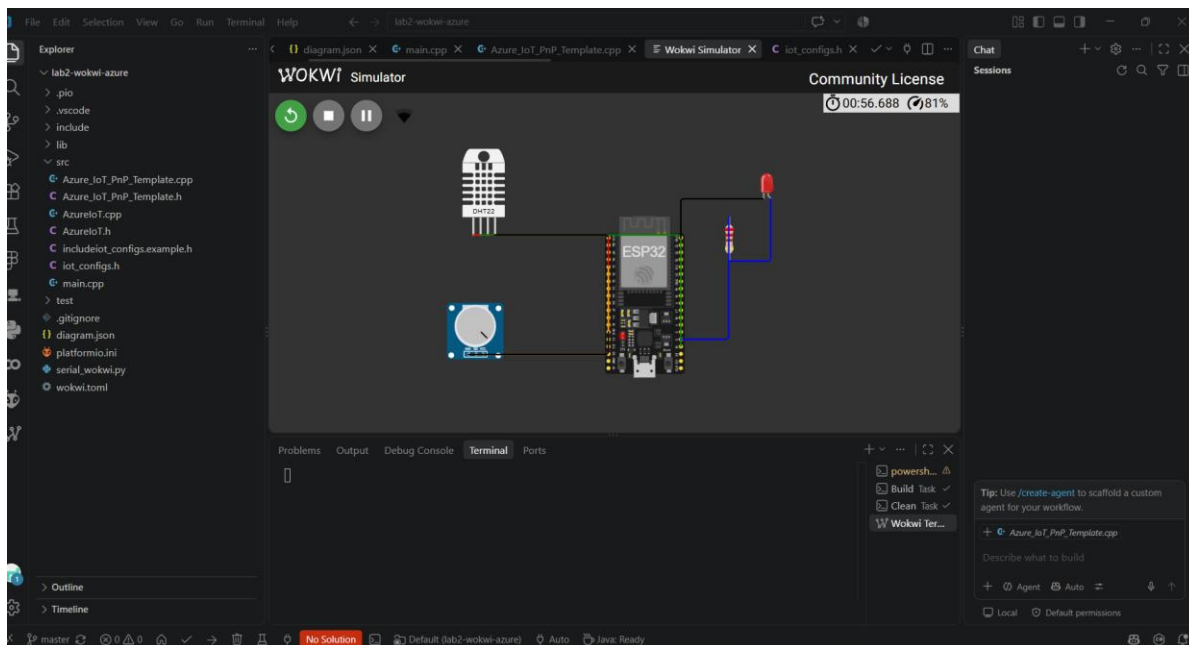


Figura 17. Proyecto PlatformIO y simulación del ESP32 en Visual Studio Code.

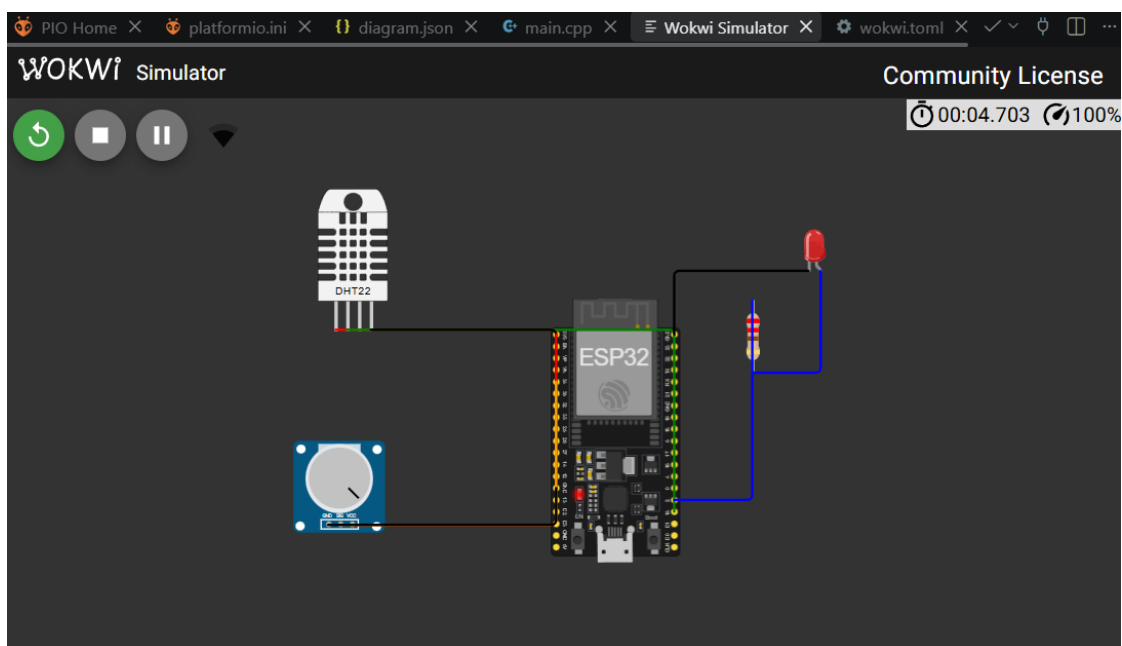


Figura 18. Circuito final del ESP32 con sensores y LED.

La siguiente evidencia muestra exclusivamente el estado inicial de Lab 2 Wokwi 08, registrado todavía sin telemetría.

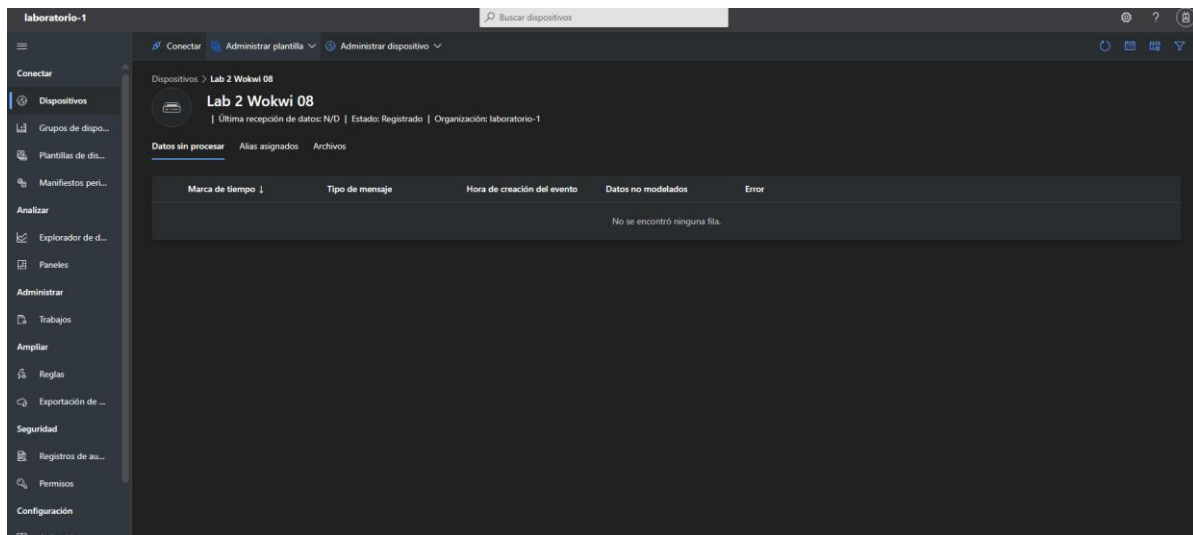


Figura 19. Lab 2 Wokwi 08 antes de recibir datos.

5.1 Telemetría del ESP32

TemperaturaDelPanel provino del DHT22. VoltajeDeSalida y CorrienteDeSalida se derivaron del potenciómetro, y PotenciaGenerada se calculó como voltaje por corriente. Todas las capturas siguientes pertenecen al dispositivo Wokwi.

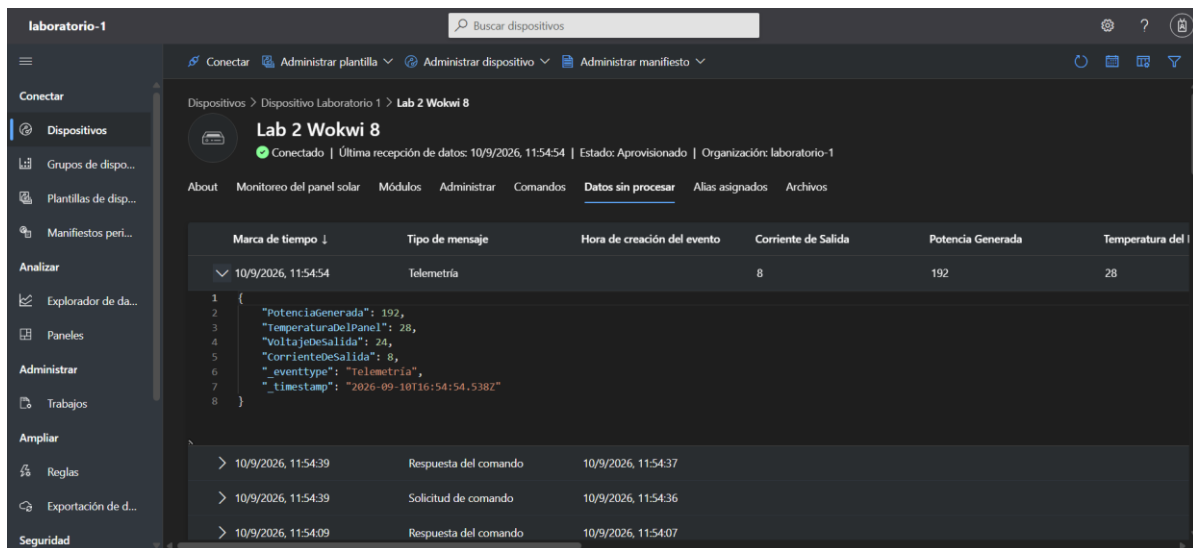


Figura 20. Datos sin procesar de Lab 2 Wokwi 8.

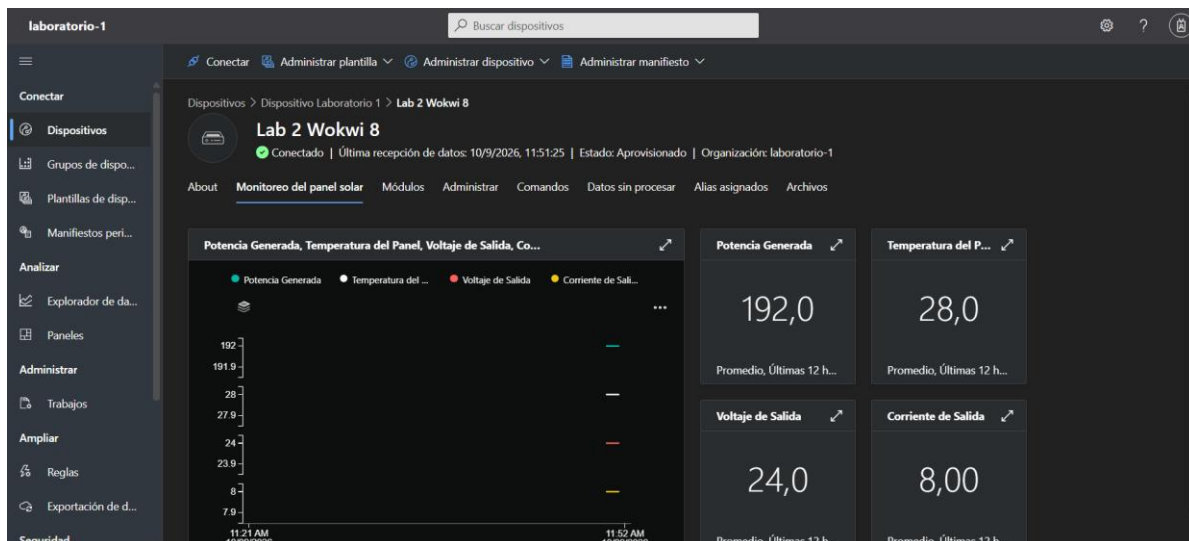


Figura 21. Panel inicial del ESP32 con temperatura de 28 °C.

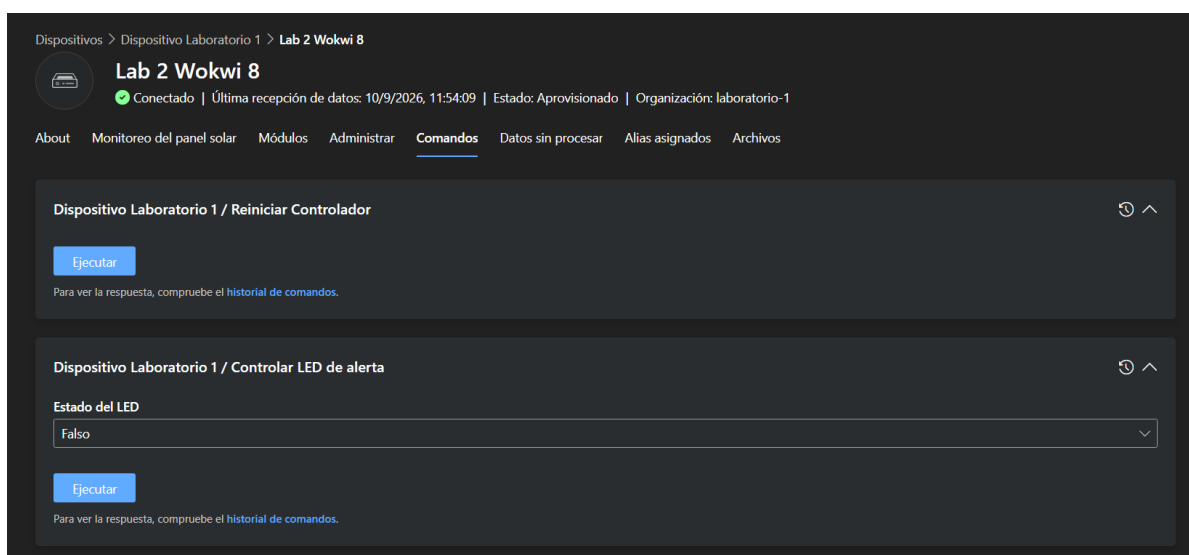


Figura 22. Lab 2 Wokwi 8 conectado, aprovisionado y con telemetría.

5.2 Control remoto del LED

ControlarLEDDeAlerta recibió un booleano. true encendió el LED y false lo apagó. El comportamiento es equivalente a setAlertLed solicitado por el enunciado.

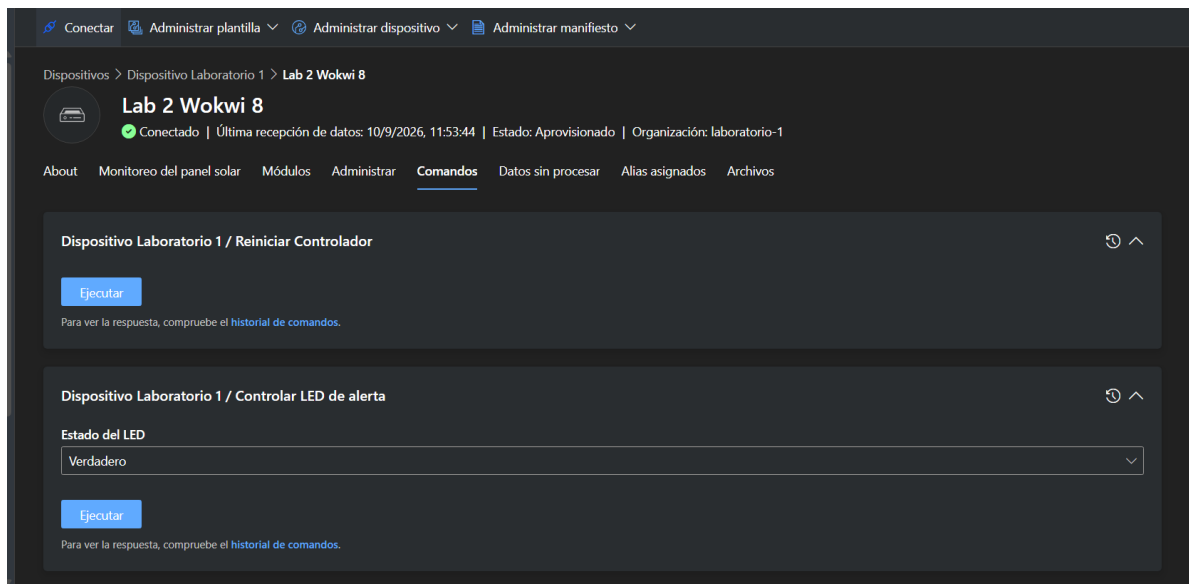


Figura 23. Envío del valor verdadero a ControlarLEDDeAlerta.

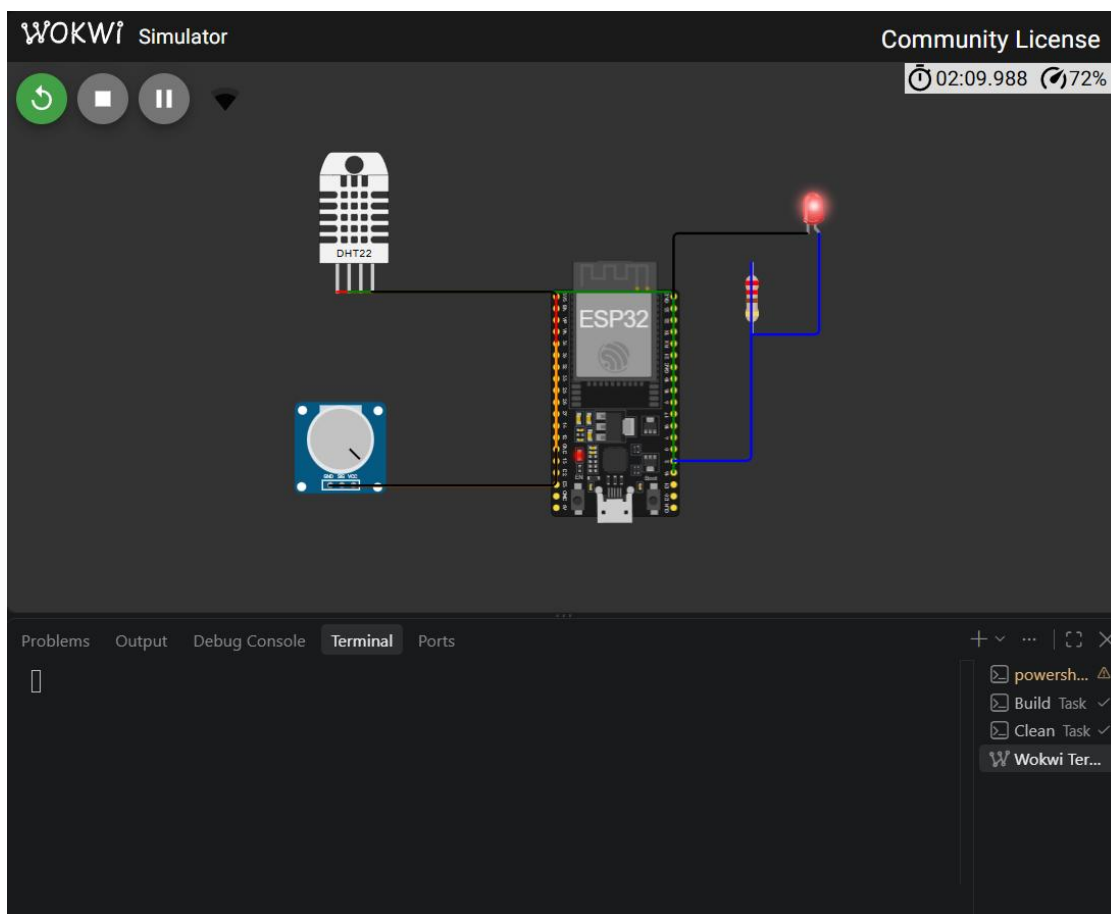


Figura 24. LED encendido después del comando.

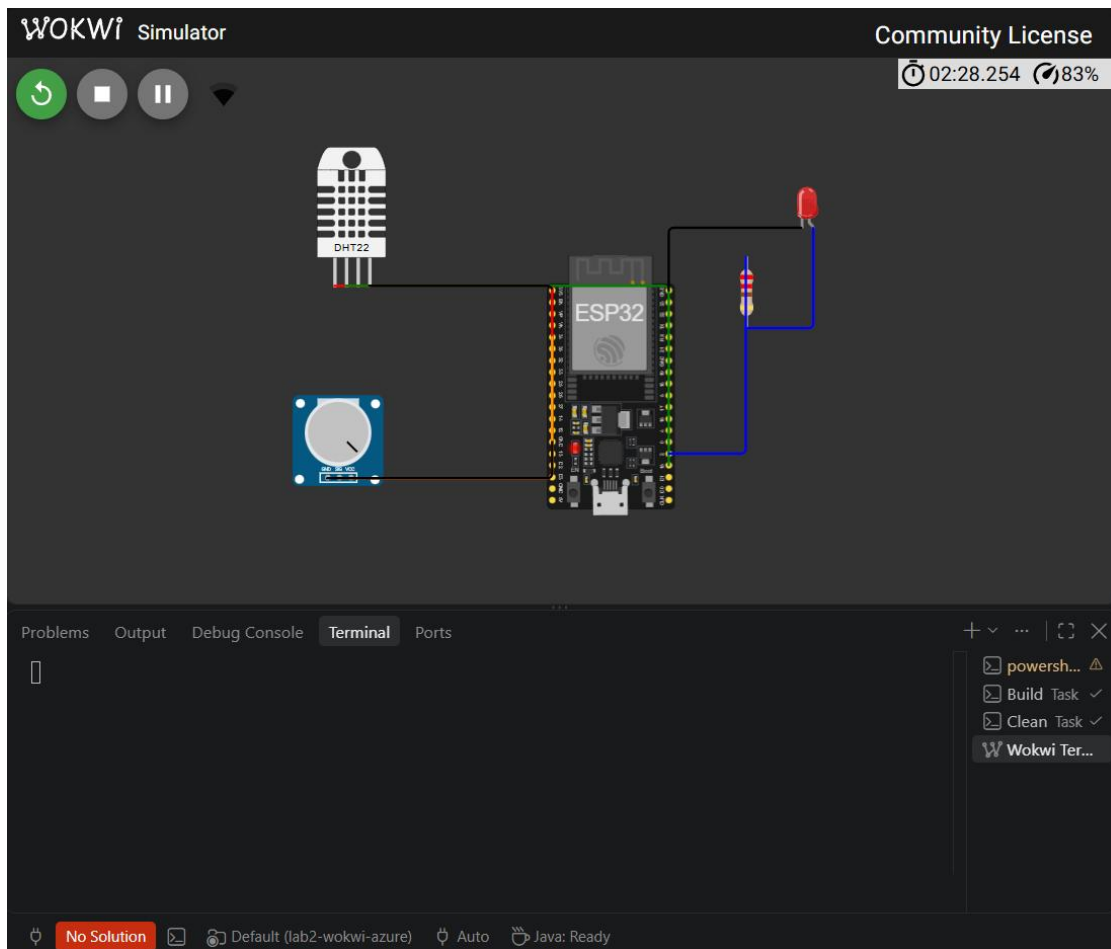


Figura 25. LED apagado después del valor falso.

6. Regla por umbral

Se comprobó una regla de temperatura alta sobre TemperaturaDelPanel. Al superar el umbral, IoT Central generó una notificación por correo. EstadoDeAlerta quedó disponible como propiedad escribible de la plantilla.

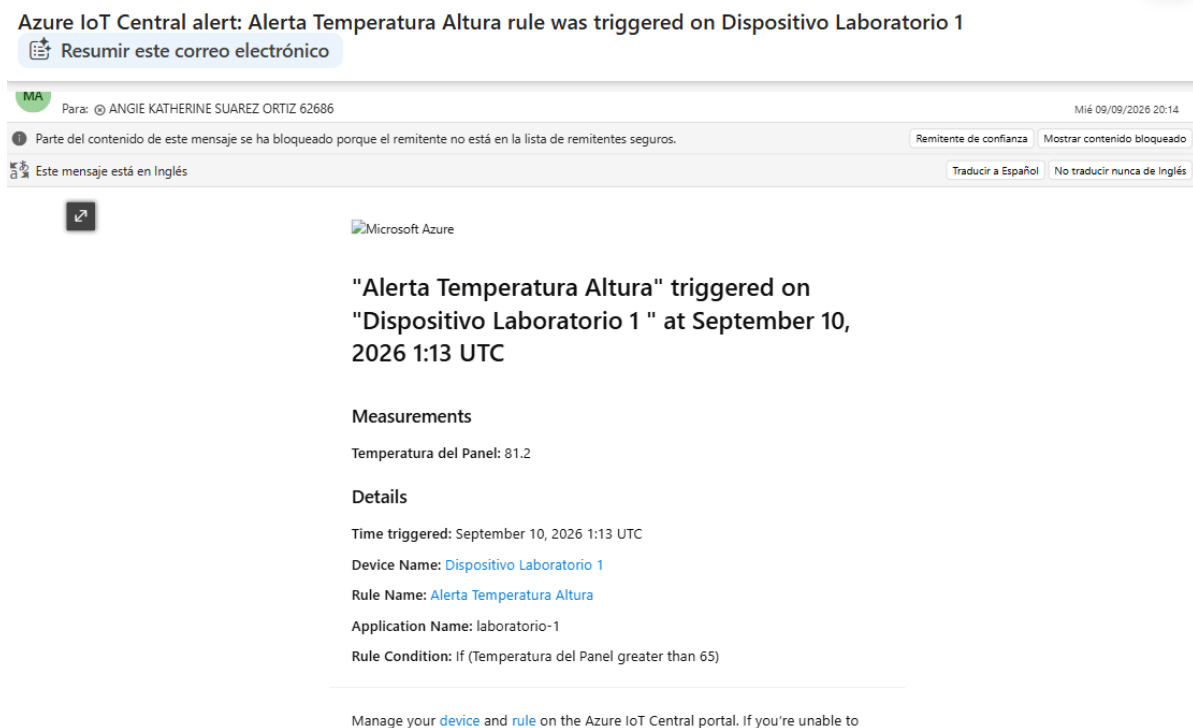


Figura 26. Correo recibido por activación de la regla de temperatura alta.

7. Comparación de métodos

Criterio	Simulador	Python en VM	ESP32 en Wokwi
Origen	Generador nativo	Valores calculados en Debian	DHT22 y potenciómetro
Configuración	Baja	Media	Alta
Depuración	Interfaz IoT Central	Terminal Python	PlatformIO y simulación
Interacción	Simulada	ReiniciarControlador	ControlarLEDDeAlerta
Fallos	Pausa del simulador	SAS, DPS y .env	Dependencias, modelo y MQTT/TLS
Realismo	Bajo	Medio	Alto dentro de la simulación

8. Resultados

- Flota de ocho dispositivos sobre una sola plantilla.
- Cuatro telemetrías enviadas por Python y Wokwi.
- ReiniciarControlador procesado por Lab 2 Python 07.
- DHT22 y potenciómetro reflejados en Lab 2 Wokwi 8.
- ControlarLEDDeAlerta con efecto visible.
- Regla de temperatura alta comprobada mediante correo.

9. Conclusiones

Azure IoT Central permitió administrar fuentes heterogéneas bajo un mismo modelo. Los simuladores facilitaron el despliegue, Python aportó control y depuración, y Wokwi representó un dispositivo embebido con sensores, actuador y conectividad segura.

La implementación confirmó la importancia de conservar coincidencia exacta entre los nombres internos de la plantilla y las claves JSON, así como de manejar correctamente SAS, DPS, Model ID y dependencias del firmware.

Se verificó comunicación bidireccional: la nube recibió telemetría desde Python y Wokwi, y ambos nodos procesaron comandos remotos. El laboratorio validó una solución IoT de extremo a extremo.